Machine Learning Course 2026 Autumn: Experiment 3

2026 Autumn

1 Overview

Implement CNNs and a small Vision Transformer on CIFAR-10. Compare their results with data augmentation, BatchNorm, dropout and positional embeddings. Complete four tasks with five training runs.

2 Dataset Description

CIFAR-10 contains 50,000 training images and 10,000 test images in 10 classes. Each image has size 32 x 32 with three colour channels.

Everyone uses the **low configuration**: **5,000 training images, 1,000 validation images and 5 epochs per run**. Use the supplied split with seed 42, batch size 64 and the full 10,000-image test set. Normalize each channel with mean 0.5 and standard deviation 0.5.

A GPU is recommended when available. The experiment can also be completed on CPU. Small models and a fixed dataset subset keep computation limited. Data are downloaded once and reused. Run the tasks in order.

3 Task Description

3.1 Task 1

Implement this three-block CNN and complete the training and evaluation functions. Use biased convolutional and linear layers and raw output logits. Tasks 1 and 2 use no BatchNorm or dropout.

Layer	Hyper-parameters	Output shape
conv_1 block	Conv 3 to 32, k=5, s=1, p=2, ReLU, AvgPool(2,2)	32 x 16 x 16
conv_2 block	Conv 32 to 64, k=3, s=1, p=1, ReLU, AvgPool(2,2)	64 x 8 x 8
conv_3 block	Conv 64 to 128, k=3, s=1, p=1, ReLU, AvgPool(2,2)	128 x 4 x 4
flatten	Keep the batch dimension	2048
fc_1	2048 to 256, then ReLU	256
fc_2	256 to 10	10

Table 1: Common CNN architecture

Train for five epochs with cross-entropy and SGD (learning rate 0.01, momentum 0.9, weight decay 0). Plot the training and validation loss and accuracy curves. The supplied helper keeps the checkpoint with the highest validation accuracy for final testing.

3.2 Task 2

Add data augmentation to the Task 1 CNN. Use RandomCrop(32, padding=4), RandomHorizontalFlip(p=0.5), ToTensor and the same normalization, in that order. Apply random transforms only to training images.

Keep the Task 1 network without BatchNorm or dropout. Train a fresh model for five epochs with the same starting weights and optimizer settings. Plot the curves and compare the test accuracies.

3.3 Task 3

Keep the convolutional layers, channel widths, pooling and fully connected layers from Task 2. Add BatchNorm between each convolution and ReLU, and dropout at the two positions shown below. Keep the augmentation, initial shared weights and training settings unchanged.

Layer	Hyper-parameters	Output shape
conv_1 block	Conv 3 to 32, k=5, s=1, p=2, BatchNorm, ReLU, AvgPool(2,2)	32 x 16 x 16
conv_2 block	Conv 32 to 64, k=3, s=1, p=1, BatchNorm, ReLU, AvgPool(2,2)	64 x 8 x 8
conv_3 block	Conv 64 to 128, k=3, s=1, p=1, BatchNorm, ReLU, AvgPool(2,2)	128 x 4 x 4
dropout	nn.Dropout(p=0.25), then flatten	2048
fc_1	2048 to 256, ReLU, nn.Dropout(p=0.25)	256
fc_2	256 to 10, raw logits	10

Table 2: Common CNN with BatchNorm and dropout

Use default affine BatchNorm2d and retain the original layer biases. The notebook reuses the base layers through inheritance. Train for five epochs and compare with Task 2 to observe the combined effect of BatchNorm and dropout.

Training and testing

Use model.train() for training. Use model.eval() and torch.no_grad() for validation and testing. Compute mean loss over all examples, including the final batch.

Keep the same data split and training settings throughout the experiment. After all five runs finish, use the shared testing cell to evaluate the validation-selected checkpoints on the test set. The test results are for the final comparison and should not be used to choose checkpoints.

The notebook provides data loading, experiment orchestration, plotting and the final comparison table. Complete the marked code and add a brief discussion of your results to the report. Extra tuning and additional training runs are not required.

3.4 Task 4

Complete a small Vision Transformer using basic PyTorch layers and tensor operations. Do not use pre-trained models or ready-made attention or Transformer modules.

Use 8 x 8 patches to reduce CPU computation. Each image produces 16 patch tokens. Prepend one class token to obtain a sequence of length 17. Complete the forward passes of attention, the encoder block and the ViT in the notebook.

Component	Specification
Patch projection	Conv2d 3 to 64, kernel=stride=8, padding=0. Flatten patches in row-major order.
Class / position	Class token (1,1,64) and position embeddings (1,17,64). Normal initialization with std=0.02.
Self-attention	4 heads, dimension 16 per head. Biased QKV and output projections. softmax(QK^T / sqrt(16)) along keys, then dropout 0.1.
Encoder block	u = x + Dropout(MHSA(LayerNorm(x))) y = u + MLP(LayerNorm(u))
MLP	Linear(64,128), GELU, Dropout(0.1), Linear(128,64), Dropout(0.1).
Output	2 encoder blocks, final LayerNorm and class-token Linear(64,10). Return raw logits.

Table 3: Small ViT architecture

Use default LayerNorm settings. There is no embedding dropout. Apply dropout to the attention output once in its residual branch. Run the provided checks before training.

Train one model with positional embeddings and one without them. Each uses five epochs, the Task 2 augmentation and AdamW (learning rate 0.001, weight decay 0.01). Keep the same initial shared weights and data split. Plot both curves and compare the five models in the final table.

4 Requirements

• Complete Tasks 1-4 in PyTorch using the fixed low configuration.

• Keep code readable and retain the notebook outputs.

• Include the training curves, comparison table and a brief discussion in your report.

• Submit the completed **.ipynb** and **report.pdf** using the supplied report template. Datasets and model files are not required.